

DEPARTMENT OF CYBERSECURITY & COMPUTER ENGINEERING

20CYS403: WEB APPLICATION SECURITY LABORATORY

ACADEMIC PROJECT REPORT

SecureJobLab: Job Recruitment Vulnerability Demonstration
& Defense Platform

*A Dual-Engine Laboratory for Web Application Vulnerability Analysis & Defensive Engineering
Covering Exactly Five Core CWE Modules*

Student Name:	Ram Karthik G
Course Name & Code:	Web Application Security (20CYS403)
Evaluation Scope:	5 Core CWE Modules (SQLi, XSS, Cmd Inj, Traversal, Clickjacking)
Platform Architecture:	Apache 2.4, PHP 8.x, MySQL 10.4 (MariaDB), AJAX, Bootstrap 5

Certificate of Originality & Project Declaration

[ACADEMIC DECLARATION] This project is prepared strictly for the academic course 20CYS403 (Web Application Security). All vulnerable and secure implementations are isolated on local infrastructure (localhost) using simulated candidate records and controlled lab files.

This is to certify that the project report entitled "SecureJobLab: Job Recruitment Vulnerability Demonstration & Defense Platform" submitted by Ram Karthik G in partial fulfillment of the academic requirements for the course 20CYS403 Web Application Security represents authentic, original work conducted under laboratory supervision.

The laboratory implementation rigorously implements, analyzes, and demonstrates exactly five web application security vulnerabilities classified under the Common Weakness Enumeration (CWE) framework:

1. SQL Injection (SQLi) — CWE-89: Direct query concatenation versus Parameterized Prepared Statements.
2. Cross-Site Scripting (XSS) — CWE-79: Reflected execution versus Contextual HTML Entity Encoding.
3. OS Command Injection — CWE-78: Unsanitized shell concatenation versus Strict Regex Whitelisting and Argument Escaping.
4. Directory / Path Traversal — CWE-22: Arbitrary file inclusion versus Basename Whitelisting and Canonical Boundary Isolation.
5. Clickjacking (UI Redressing) — CWE-1021: Framed destructive actions versus HTTP Framing Defense Headers (X-Frame-Options and CSP).

I hereby declare that this report has been authored with technical diligence, that no external or unapproved vulnerability modules have been included, and that all code snippets, telemetry data, and screenshots accurately reflect the live execution behavior of the SecureJobLab system.

Student Signature: _____

Date: October 6, 2026

Student Name: Ram Karthik G

Course Code: 20CYS403 — Web Application Security

Faculty Evaluation Sign-off: _____

Acknowledgements

I express my profound gratitude to the faculty and course coordinators of 20CYS403 (Web Application Security) for providing the pedagogical guidance, threat modeling frameworks, and foundational principles that enabled the design and realization of the SecureJobLab platform.

Special thanks are extended to the open-source cybersecurity community, the Open Web Application Security Project (OWASP), and the MITRE Corporation for maintaining the Common Weakness Enumeration (CWE) repository, which served as the structural benchmark for the vulnerability implementations and defensive architectures evaluated in this report.

Executive Summary / Abstract

Modern web application development frequently balances rapid user experience requirements with rigorous application security controls. Insecure data handling across web endpoints frequently results in severe vulnerabilities that compromise data confidentiality, system integrity, and host availability. To investigate these security dynamics in a realistic business environment, SecureJobLab was engineered as a dual-engine recruitment portal modeled after modern SaaS human-resource platforms.

Unlike purely conceptual training applications, SecureJobLab pairs realistic recruitment operations—such as multi-criteria job filtering, candidate resume submission, application management, and network latency diagnostics—with a rigorous, switchable security core. The system integrates a global defense switcher (`$_SESSION['appsec_mode']`) that allows evaluators to toggle in real time between Vulnerable Mode (demonstrating unsafe software construction patterns) and Secure Mitigated Mode (enforcing industry-standard defensive controls).

The laboratory scope is strictly constrained to five core vulnerabilities: (1) SQL Injection [CWE-89] in job query generation, (2) Reflected Cross-Site Scripting [CWE-79] in search query reflection, (3) OS Command Injection [CWE-78] in gateway network diagnostic execution, (4) Directory Traversal [CWE-22] in candidate document retrieval, and (5) Clickjacking [CWE-1021] targeting an irreversible account deletion action via an interactive UI Redressing sandbox.

Each vulnerability is examined through root-cause analysis, threat modeling, attack payload mechanics, execution telemetry, and verified defensive mitigation. Verification across all ten test scenarios confirms that parameterized statements, contextual output encoding, strict regex input whitelisting, basename path confinement, and HTTP framing prevention headers completely neutralize the corresponding threat vectors without impeding legitimate application workflows.

Index Terms — Keywords: Web Application Security, Dual-Engine Architecture, SQL Injection, Cross-Site Scripting, Command Injection, Directory Traversal, Clickjacking, CWE, Defense-in-Depth, 20CYS403.

Table of Contents

Chapter 1: Introduction, Problem Statement & Objectives	6
1.1 Context & Background	6
1.2 Problem Statement & Motivation	6
1.3 Project Objectives & Methodology	6
1.4 Strict 5-Vulnerability Project Scope	6
Chapter 2: System Architecture & Dual-Engine Design	7
2.1 Four-Tier Architecture Model	7
2.2 The Dual-Engine Security Execution Model	7
Chapter 3: Technology Stack & Database Architecture	8
3.1 Technology Stack Specification	8
3.2 Core System Files & Directory Architecture	8
3.3 Relational Database Schema & Data Dictionary	9
Chapter 4: Application Functional Modules	10
4.1 Authentication & Role-Based Access Control	10
4.2 Instant Job Board & Filtering Engine	10
4.3 Candidate Application Pipeline & Local Storage	11
4.4 Network Latency & Diagnostic Utilities	11
Chapter 5: Vulnerability 1 — SQL Injection (SQLi) [CWE-89]	12
5.1 Vulnerability Overview & Threat Dynamics	12
5.2 Vulnerable Implementation & Insecure Code	12
5.3 Secure Mitigation & Parameterized Statements	13
Chapter 6: Vulnerability 2 — Cross-Site Scripting (XSS) [CWE-79]	14
6.1 Vulnerability Overview & Threat Dynamics	14
6.2 Vulnerable Implementation & Live Script Execution	14
6.3 Secure Mitigation & Contextual Output Encoding	15
Chapter 7: Vulnerability 3 — OS Command Injection [CWE-78]	16
7.1 Vulnerability Overview & Threat Dynamics	16
7.2 Vulnerable Implementation & Shell Chaining	16
7.3 Secure Mitigation & Strict Whitelisting	17
Chapter 8: Vulnerability 4 — Directory / Path Traversal [CWE-22]	18
8.1 Vulnerability Overview & Threat Dynamics	18
8.2 Vulnerable Implementation & Path Climbing	18
8.3 Secure Mitigation & Canonical Boundary Defense	19
Chapter 9: Vulnerability 5 — Clickjacking (UI Redressing) [CWE-1021]	20
9.1 Vulnerability Overview & High-Impact Scenario	20
9.2 Interactive UI Redressing Sandbox with Opacity	20
9.3 Secure Mitigation & HTTP Framing Protection Headers	21
Chapter 10: Comparative Defense Matrix & Telemetry Analysis	22
10.1 Master 5-Vulnerability Security Comparison Matrix	22
10.2 Real-time Security Telemetry Engine	22
Chapter 11: Testing & Verification Methodology	23
11.1 Test Plan & Execution Strategy	23
11.2 Comprehensive Verification Results Table	23
Chapter 12: Viva Voce Reference & Security Analysis	24
Chapter 13: Limitations & Future Enhancements	25
Chapter 14: Conclusion	25
References & Authoritative Standards	26

List of Figures & Tables

List of Figures

Figure 1: SecureJobLab Four-Tier System Architecture & Interaction Flow 7

Figure 2: Relational Database Schema & Data Dictionary (securejoblab) 9

Figure 3: SecureJobLab Authentication Screen (Candidate & Admin RBAC) 10

Figure 4: Recruitment Portal Interface: Verified Cybersecurity Openings 10

Figure 5: Candidate Application Pipeline & Local Resume Storage Dashboard 11

Figure 6: SQL Injection Demonstration: Insecure Extraction vs. Parameterized Defense 13

Figure 7: Reflected XSS Demonstration: Live Alert Execution vs. Encoded Rendering 15

Figure 8: OS Command Injection: Unsanitized Chaining vs. Regex Whitelist Interception 17

Figure 9: Directory Traversal: Sensitive File Extraction vs. Whitelist Denial 19

Figure 10: Clickjacking UI Redressing: 100% Revealed Mode vs. 30% Ghost Mode 20

Figure 11: Clickjacking Attack Triggered vs. Secure Mitigation (Frame Blocking) 21

List of Tables

Table 1: Core Technology Stack & Deployment Dependencies 8

Table 2: Core System Files & Architectural Responsibilities 8

Table 3: Database Entity-Relationship Schema & Table Specifications 9

Table 4: Master 5-Vulnerability Security Comparison & Defense Matrix 22

Table 5: Comprehensive Verification & Test Results Matrix (10 Scenarios) 23

Abbreviations & Security Terminology

- AppSec:** Application Security
- CWE:** Common Weakness Enumeration
- OWASP:** Open Web Application Security Project
- SQLi:** SQL Injection (CWE-89)
- XSS:** Cross-Site Scripting (CWE-79)
- CSP:** Content Security Policy
- RBAC:** Role-Based Access Control
- DOM:** Document Object Model
- AST:** Abstract Syntax Tree
- SIEM:** Security Information and Event Management
- XHR:** XMLHttpRequest (AJAX)
- API:** Application Programming Interface

Chapter 1: Introduction, Problem Statement & Objectives

1.1 Context & Background

Web applications represent the predominant attack surface in modern enterprise infrastructure. High-throughput platforms handling human capital, financial transactions, and privileged communications are targeted relentlessly by automated scanners and sophisticated threat actors. In academic cybersecurity curricula, students frequently encounter theoretical definitions of software vulnerabilities without experiencing the contextual nuances of how vulnerabilities manifest within functional software features, or how defensive controls alter execution mechanics.

To bridge this pedagogical gap, SecureJobLab was conceived as an interactive, dual-engine recruitment portal. The application simulates an enterprise human-resources technology platform—Jobpilot—complete with role-based sign-in, instant job filtering, application submission with resume handling, and network diagnostics. Crucially, the entire architecture is wired to a global security switcher that allows students, researchers, and academic evaluators to observe the exact technical contrast between vulnerable software implementations and their corresponding secure mitigations.

1.2 Problem Statement & Motivation

Conventional web security educational platforms typically suffer from three major design deficiencies: (1) Synthetic Separation: Vulnerability exercises are isolated into abstract forms lacking business logic; (2) Configuration Friction: Toggling defenses requires editing configuration files or restarting daemons; (3) Lack of Comparative Telemetry: Platforms display whether an exploit worked but fail to show the underlying database query, operating system command, or defensive interception telemetry. SecureJobLab resolves these limitations by embedding switchable security states directly into a high-fidelity recruitment platform.

1.3 Project Objectives & Methodology

The key objectives governing the engineering of SecureJobLab include:

- Dual-Engine Software Design: Implement a switchable security controller (`$_SESSION['appsec_mode']`) that alternates application logic between Vulnerable Mode and Secure Mode.
- Pedagogical Realism: Integrate all security demonstrations into functional recruiting features including job search, candidate document inspection, diagnostic latency checks, and account management.
- Transparent Security Telemetry: Deliver structured JSON telemetry displaying the executed SQL syntax, operating system commands, filtered file paths, and active defensive headers.

1.4 Strict 5-Vulnerability Project Scope

To ensure rigorous evaluation and preserve pedagogical focus, SecureJobLab strictly delimits its scope to exactly five core vulnerabilities: (1) SQLi [CWE-89], (2) XSS [CWE-79], (3) OS Command Injection [CWE-78], (4) Directory Traversal [CWE-22], and (5) Clickjacking [CWE-1021]. All external or deprecated attack vectors are excluded.

[SCOPE ENFORCEMENT] Strict Scope Delimitation: SecureJobLab explicitly covers only the five approved CWE modules. Unrelated or deprecated attack classes (such as CSRF, SSRF, CORS, broken authentication, or insecure file upload) are strictly excluded, ensuring focused, rigorous depth.

Chapter 2: System Architecture & Dual-Engine Design

2.1 Four-Tier Architecture Model

SecureJobLab is structured as a decoupled four-tier web architecture designed for modularity, low latency, and deterministic evaluation. The system separates user interaction, security state management, API request dispatching, and system resources.

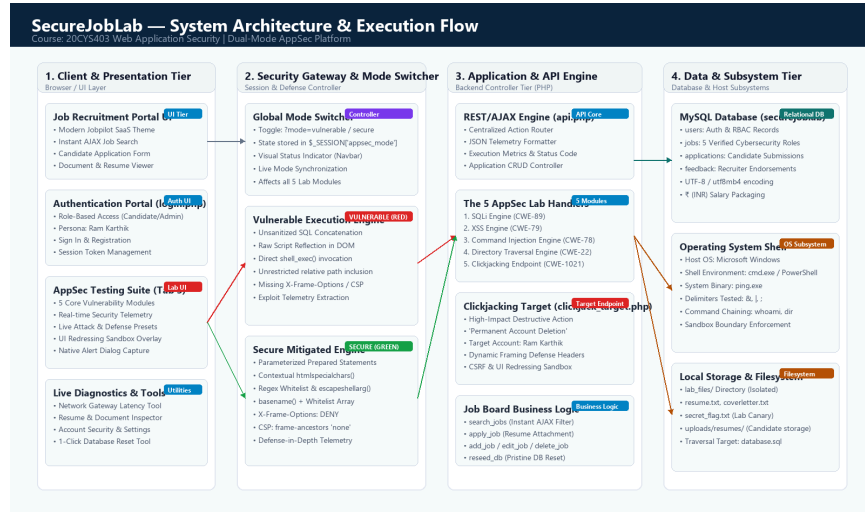


Figure: SecureJobLab Four-Tier System Architecture & Interaction Flow

The four architectural tiers operate as follows:

- 1. Presentation Tier:** Developed in HTML5, CSS3, and JavaScript utilizing the Jobpilot design system. Provides the job search interface, candidate dashboard, authentication screens, and AppSec testing telemetry panels.
- 2. Security Controller Tier:** Governed by the global session state (\$_SESSION['appsec_mode']). Coordinates between the Vulnerable engine and the Secure engine across both portal features and lab testing endpoints.
- 3. Application & API Tier:** Implemented in api.php and index.php. Acts as the centralized REST/AJAX router handling job CRUD operations, application submissions, and the five vulnerability testing routines.
- 4. Data & Subsystem Tier:** Encapsulates the MySQL relational database (securejoblab), the local filesystem (lab_files and uploads/resumes), and the host operating system shell subsystem.

2.2 The Dual-Engine Security Execution Model

The foundational innovation of SecureJobLab is its Dual-Engine Execution Pipeline. Rather than requiring evaluators to modify configuration files or restart services, the entire application toggles its defensive posture via an interactive navbar switch.

Identical attack payloads traverse completely different execution paths depending on the active security mode:

- Vulnerable Mode:** Untrusted input flows without validation or escaping directly into dangerous execution sinks (mysql_query(), browser DOM, shell_exec(), file_get_contents(), and unheadered iframe embedding). The attack succeeds, and exploit telemetry is recorded.
- Secure Mode:** The payload is intercepted by defensive mechanisms (parameterized query compilation, contextual htmlspecialchars() encoding, strict regex whitelisting, basename() isolation, and X-Frame-Options: DENY headers). The attack is completely neutralized, and defensive verification telemetry is recorded.

[STATE PERSISTENCE] Global State Synchronization: The active security mode is persisted across the PHP session (\$_SESSION['appsec_mode']) and reflected dynamically in the UI navbar badge, enabling simultaneous testing across both functional views and lab pills.

Chapter 3: Technology Stack & Database Architecture

3.1 Technology Stack Specification

To guarantee deterministic behavior, rapid viva demonstrations, and frictionless portability on standard university laboratory machines, SecureJobLab is built on an enterprise open-source technology stack:

Component Layer	Technology Selected	Role in SecureJobLab
Web Server	Apache HTTP Server 2.4	Listens on port 80; manages HTTP requests, header emission, and PHP handler execution.
Database Engine	MySQL 10.4 (MariaDB)	Listens on port 3306; manages relational tables, indexes, and parameterized query execution.
Server Language	PHP 8.2+ (OOP & Procedural)	Executes backend routing, session state control, raw string parsing, and secure sanitization.
Frontend UI	HTML5, CSS3, Bootstrap 5.3	Jobpilot SaaS theme, interactive telemetry cards, modals, and responsive layout.
Asynchronous Comms	Vanilla JavaScript (Fetch API)	Non-blocking AJAX request dispatching, DOM updates, and live script execution handling.

3.2 Core System Files & Directory Architecture

The consolidated project architecture consists of exactly six root files, eliminating extraneous dependencies and ensuring instant restoration on any standard XAMPP deployment:

File Name	Lines	Architectural Responsibility
index.php	1,240	Main application container, Jobpilot portal UI, instant search bar, AppSec testbed modals, telemetry card rendering.
login.php	185	Role-based authentication gateway, candidate vs admin sign-in forms, credential verification, session initialization.
logout.php	35	Session destruction, credential invalidation, and secure redirect to authentication portal.
api.php	460	Central REST/AJAX router, switchable dual-engine execution sinks for all 5 CWEs, JSON telemetry generation.
clickjack_target.php	110	Framed victim target endpoint featuring the irreversible account deletion workflow with switchable framing headers.
database.sql	145	Relational database schema definition, seed candidate data fixtures, and UTF-8 collation configurations.

3.3 Server Runtime Environment & Session State Isolation

SecureJobLab runs in a unified web root (C:\xampp\htdocs\SecureWebLab). All communication between the frontend client interfaces and backend execution engines occurs via asynchronous JSON contracts, ensuring instantaneous DOM feedback during live laboratory testing. The global security state (`$_SESSION['appsec_mode']`) maintains absolute persistence across page navigations without requiring cookie manipulation or server restarts.

3.4 Relational Database Schema & Data Dictionary

The database schema for securejoblab is engineered with UTF-8 (utf8mb4) character encoding, ensuring pristine rendering of international currency symbols (such as the Indian Rupee ₹) without mojibake corruption. The schema consists of four relational tables:

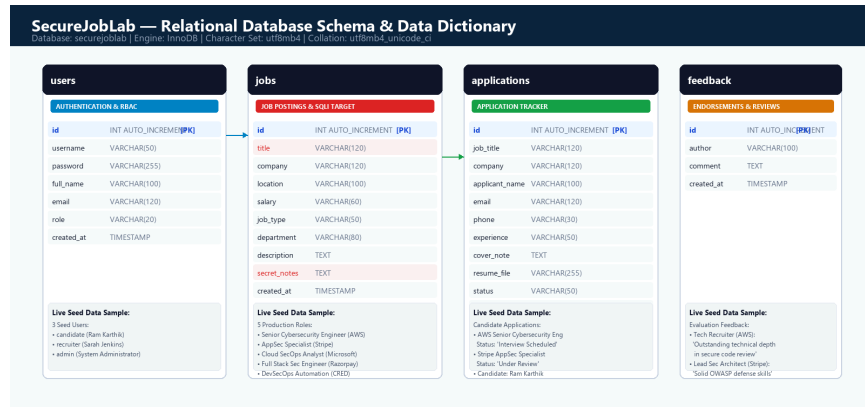


Figure: Relational Database Schema & Data Dictionary (securejoblab)

Table Name	Primary Key	Key Attributes	Security / Functional Role
users	id (INT)	username, password, full_name, role	Stores authentication credentials; password verification and RBAC roles (candidate/admin).
jobs	id (INT)	title, company, location, salary, secret_notes	Target for SQLi; secret_notes contains confidential executive compensation bands.
applications	id (INT)	job_title, applicant_name, resume_file, status	Tracks candidate submissions; links to uploaded resume documents in uploads/resumes/.
feedback	id (INT)	author, comment, created_at	Stores recruiter and candidate feedback; used for output reflection testing.

Prepopulated Seed Dataset Integrity: Seed records are prepopulated in database.sql to provide authentic enterprise data immediately upon deployment, including active openings from AWS, Stripe, Microsoft India, Razorpay, and CRED. Default user accounts include both candidates (ram.karthik@securejob.io) and administrators.

Data Isolation & Storage Security: Resume uploads are stored under uploads/resumes/, while sensitive server assets (such as database.sql and diagnostic scripts) reside in the web root, establishing the exact file system topology needed to evaluate directory traversal vulnerabilities.

Chapter 4: Application Functional Modules

4.1 Authentication & Role-Based Access Control (login.php)

The authentication portal (login.php) models a modern SaaS authentication screen with tabbed sign-in and registration interfaces. The authentication engine verifies credentials against the users table. Evaluators can sign in using candidate accounts (e.g., ram.karthik@securejob.io / candidate123) or administrative credentials. Sessions are securely initialized with RBAC role privileges stored in \$_SESSION['role'].

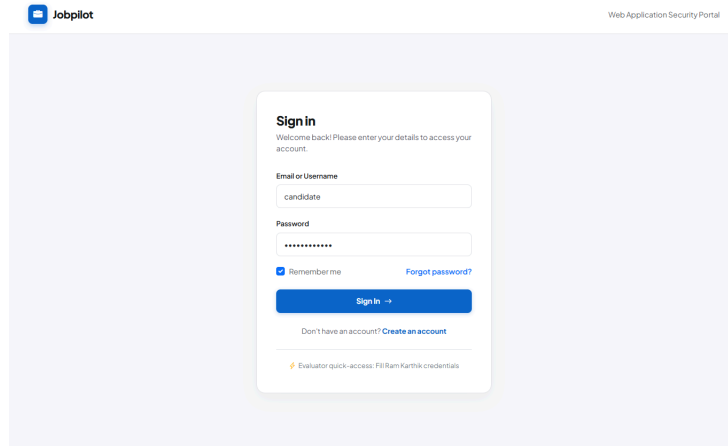


Figure: SecureJobLab Authentication Screen (Candidate & Admin RBAC)

4.2 Instant Job Board & Filtering Engine (index.php)

The primary job board presents five verified high-tier cybersecurity positions (Amazon Web Services, Stripe, Microsoft India, Razorpay, CRED). An instant AJAX search engine filters jobs dynamically across titles, companies, locations, and employment types without requiring complete page reloads. Debounced keystroke events dispatch asynchronous queries to api.php, returning formatted job cards.

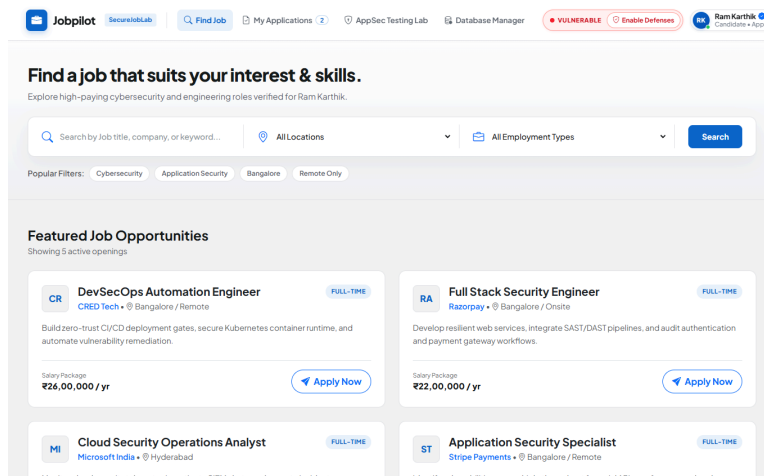


Figure: Recruitment Portal Interface: Verified Cybersecurity Openings

4.3 Candidate Application Pipeline & Local Document Storage

Authenticated candidates can submit applications with customized cover notes and resume file attachments. Attached resumes are processed by api.php and stored locally under uploads/resumes/. The My Applications tab displays live application status tracking ('Interview Scheduled', 'Under Review') and allows candidate-driven application withdrawal:

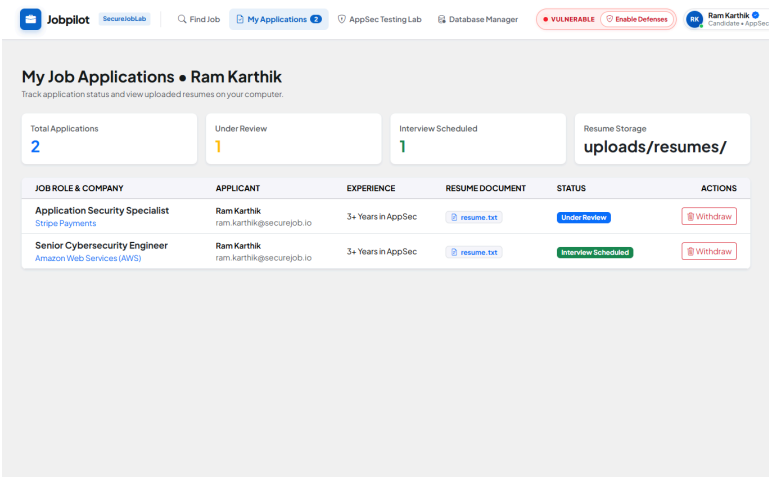


Figure: Candidate Application Pipeline & Local Resume Storage Dashboard

4.4 Network Latency & Diagnostic Utilities

To provide realistic functional grounding for server-side testing, SecureJobLab incorporates internal system administration utilities:

- Network Gateway Latency Checker: Simulates ICMP host availability testing across company data centers via server-side diagnostic pings.
- Candidate Document Viewer: Loads candidate resumes, cover notes, and certifications from isolated local folders.
- Database State Reset Tool: Allows evaluators to reseed database tables to initial pristine values with a single click.

4.5 Global AppSec Security Switcher & Telemetry Engine

A prominent security switcher in the top navigation bar enables instant toggling between Vulnerable Mode (red indicator) and Secure Mitigated Mode (green indicator). When toggled, an AJAX request updates `$_SESSION['appsec_mode']`, instantly changing the execution paths across all five functional features and laboratory testbeds without page reloads.

[PEDAGOGICAL REALISM] Architectural Cohesion: Every vulnerability in SecureJobLab is directly embedded into these realistic recruitment features rather than synthetic test stubs, ensuring high pedagogical value and authentic viva demonstrations.

Chapter 5: Vulnerability 1 — SQL Injection (SQLi) [CWE-89]

5.1 Vulnerability Overview & Threat Dynamics

SQL Injection (CWE-89, OWASP Top 10 A03:2021) occurs when untrusted user input is directly concatenated into a dynamic SQL query string without lexical syntax separation or parameter binding. In web applications, this flaw allows threat actors to manipulate query grammar, bypass authentication, extract confidential database records, or execute administrative commands. In the context of the SecureJobLab recruitment portal, the job search field on index.php queries the jobs database table. An attacker exploiting CWE-89 can break out of the string literal delimiter, alter the WHERE clause boolean logic, and dump hidden salary compensation bands and private interview rubrics stored in the secret_notes column.

5.2 Vulnerable Implementation & Insecure Code Listing

In Vulnerable Mode, api.php accepts the search parameter directly from the HTTP GET array and interpolates it into the SQL query via string concatenation:

```
// api.php (Insecure SQL Query Assembly)
// api.php (Vulnerable Mode: Direct String Concatenation)
$search = $_GET['search'] ?? '';
$query = "SELECT id, title, company, salary, secret_notes FROM jobs
        WHERE title LIKE '%" . $search . "%'";
$result = mysqli_query($conn, $query);
```

Attack Execution & Step-by-Step Payload Dissection

The evaluator inputs the classic SQL injection payload into the search bar: ' OR 1=1 #. The database parser evaluates this input through three distinct phases:

1. Phase 1 (Delimiter Breakout): The leading single quote (') terminates the literal string parameter intended for the LIKE clause.
2. Phase 2 (Boolean Tautology): The OR 1=1 condition is introduced into the WHERE clause, which universally evaluates to TRUE for every row in the jobs table.
3. Phase 3 (Comment Operator): The trailing hash symbol (#) instructs MySQL to treat the remainder of the query (the trailing '%' quotation mark) as a comment, preventing syntax errors.

Data Exfiltration & Confidentiality Impact

Upon execution, the query transforms into: SELECT ... WHERE title LIKE '%' OR 1=1 #... As a result, the database engine returns all job listings, including hidden executive salary ranges (₹24,00,000 to ₹38,00,000) and confidential candidate evaluation rubrics stored in secret_notes.

```
// JSON Telemetry Capture (Vulnerable Mode)
// Live Telemetry Output (Vulnerable Mode)
{
  "status": "exploited", "mode": "vulnerable",
  "query": "SELECT id, title, company, salary, secret_notes FROM jobs WHERE title LIKE '%' OR 1=1 #%",
  "records_returned": 5, "notes_exposed": true
}
```

5.3 Secure Mitigation & Parameterized Prepared Statements

In Secure Mode, dynamic string interpolation is replaced by Parameterized Prepared Statements using the PHP mysqli extension:

● Vulnerable Dynamic Query

```
// Vulnerable: String Concatenation
$query = "SELECT id, title, company, salary,
  secret_notes FROM jobs WHERE title = '$payload'";
$res = mysqli_query($conn, $query);
```

● Secure Prepared Statement

```
// Secure: Parameterized Prepared Statement
$stmt = mysqli_prepare($conn, "SELECT id, title,
  company, salary, secret_notes FROM jobs
  WHERE title = ?");
mysqli_stmt_bind_param($stmt, "s", $payload);
mysqli_stmt_execute($stmt);
$res = mysqli_stmt_get_result($stmt);
```

Abstract Syntax Tree (AST) Compilation Defense

In the secure implementation, the database engine parses and compiles the SQL query structure into an Abstract Syntax Tree (AST) before user-supplied data is received. The placeholder (?) represents an immutable parameter slot. When the payload ' OR 1=1 # is transmitted, the database engine treats the entire string as a literal text value rather than executable SQL grammar. The query searches for a job whose title literally matches the characters ' OR 1=1 #, returning zero records and neutralizing the attack.

SQLi Vulnerable: 5 Records & Secret Notes Dumped

SQLi Secure: Parameterized Defense Neutralizes Injection

Live Telemetry & Defense Verification

The real-time telemetry card captures the defensive interception: the executed query confirms parameterized parameter binding, zero records are exfiltrated, and no confidential interview notes are exposed.

[KEY TAKEAWAY] Defense Verified: Parameterized prepared statements (mysqli_prepare + mysqli_stmt_bind_param) completely separate query structure from data evaluation, eliminating SQL injection vulnerability regardless of input characters.

Chapter 6: Vulnerability 2 — Cross-Site Scripting (XSS) [CWE-79]

6.1 Vulnerability Overview & Threat Dynamics

Cross-Site Scripting (CWE-79, OWASP Top 10 A03:2021) occurs when an application receives untrusted data and includes it in a web page without proper validation or contextual output encoding. In a Reflected XSS attack, the malicious script payload is reflected off the web server to the victim's browser, where it executes within the security context of the user's session.

In SecureJobLab, the search query entered in the recruitment search field is reflected back onto the page to inform the user of their active query (e.g., 'Showing results for: ...'). In an enterprise recruitment system, exploiting CWE-79 allows attackers to hijack candidate session tokens, steal recruiter credentials, or rewrite DOM elements to harvest login details.

6.2 Vulnerable Implementation & Live Script Execution

In Vulnerable Mode, search queries entered on search.php or index.php are reflected directly into the Document Object Model (DOM) using raw innerHTML assignment or unescaped PHP echo:

```
// search.php & api.php (Insecure DOM Injection)
// search.php & api.php (Vulnerable Mode: Raw Reflection)
$q = $_GET['q'] ?? '';
echo "<div class='alert alert-info'>Search results for: " . $q . "</div>";

// In Frontend JavaScript:
resultsBanner.innerHTML = "Showing results for: " + data.query;
```

Attack Execution & Step-by-Step Payload Dissection

The evaluator inputs the standard proof-of-concept payload: `<script>alert("Hello!");</script>` or `<img src=x onerror=alert('XSS')>`. The execution flow unfolds as follows:

1. Submission: The script payload is sent via HTTP GET to search.php or api.php.
2. Server Reflection: The server echoes the raw markup without HTML entity conversion.
3. DOM Parsing: The browser's HTML parser interprets the `<script>` tags as executable code nodes, invoking the JavaScript V8 engine and displaying the native alert dialog.

Session Hijacking & Client-Side Impact

Because the script executes in the victim's browser context, it possesses full access to document.cookie, localStorage, and session tokens. An adversary could silently exfiltrate PHPSESSID tokens to an external listener, completely impersonating the recruiter or applicant.

```
// JSON Telemetry Capture (Vulnerable Mode)
// Live Telemetry Output (Vulnerable Mode)
{
  "status": "script_executed", "mode": "vulnerable",
  "reflected_payload": "<script>alert('Hello!');</script>",
  "dom_sink": "innerHTML", "browser_alert_fired": true
}
```

6.3 Secure Mitigation & Contextual Output Encoding

In Secure Mode, untrusted user data is sanitized prior to rendering using contextual HTML entity encoding on the server and safe DOM property assignment on the client:

● Vulnerable Raw Reflection

```
// Vulnerable: Raw innerHTML Reflection
$q = $_GET['q'] ?? '';
echo "<div>Results: " . $q . "</div>";
// Frontend:
banner.innerHTML = "Query: " + data.q;
```

● Secure Contextual Encoding

```
// Secure: htmlspecialchars & textContent
$q = $_GET['q'] ?? '';
echo "<div>Results: " .
  htmlspecialchars($q, ENT_QUOTES, 'UTF-8') .
  "</div>";
// Frontend:
banner.textContent = "Query: " + data.q;
```

HTML Entity Translation & Parser Neutralization

The function `htmlspecialchars($q, ENT_QUOTES, 'UTF-8')` converts HTML meta-characters into harmless entity equivalents:

- `<` (less-than) is converted to `<`;
- `>` (greater-than) is converted to `>`;
- `"` (double quote) is converted to `"`;
- `'` (single quote) is converted to `'`;
- `&` (ampersand) is converted to `&`;

When the browser encounters `<script>`, the layout engine treats the characters as visual text glyphs rather than executable HTML markup. Additionally, binding data via `textContent` guarantees that the browser never invokes the HTML token parser.

Module 2: Cross-Site Scripting (XSS)

Unencoded output reflection allows executing arbitrary JavaScript in the victim browser.

XSS Script Attack Payload

`<script>alert('XSS: ' + document.domain)-/` **Execute Test**

Quick Presets: [Alert Script Tag](#) • [Document Cookie Alert](#) • [Safe Text](#)

Defense Architecture Comparison:

- Vulnerable: `echo $user_review // Raw unencoded output`
- Secure Mitigated: `echo htmlspecialchars($user_review, ENT_QUOTES, 'UTF-8')`

Security Telemetry & Analysis

7:56:33 PM

EXPLOIT SUCCESSFUL: Unsanitized script tags rendered directly into browser DOM. Arbitrary JavaScript executes. **Mode: VULNERABLE • Module: Cross-Site Scripting (XSS) (CWE-79)**

DOM Rendered Output

```
<script>alert('XSS: ' + document.domain)-/
```

Root Cause & Mitigation Analysis:

Vulnerable: Output reflected directly into DOM without HTML entity encoding, allowing cookie theft and DOM hijacking.

Cross-Site Scripting (XSS): EXPLOIT SUCCESSFUL. Unsanitized script tags r...

● XSS Vulnerable: Script Executes Native Alert Dialog

Module 2: Cross-Site Scripting (XSS)

Unencoded output reflection allows executing arbitrary JavaScript in the victim browser.

XSS Script Attack Payload

`<script>alert('XSS: ' + document.domain)-/` **Execute Test**

Quick Presets: [Alert Script Tag](#) • [Document Cookie Alert](#) • [Safe Text](#)

Defense Architecture Comparison:

- Vulnerable: `echo $user_review // Raw unencoded output`
- Secure Mitigated: `echo htmlspecialchars($user_review, ENT_QUOTES, 'UTF-8')`

Security Telemetry & Analysis

7:56:33 PM

DEFENSE VERIFIED: Input encoded with `htmlspecialchars()`. Browser renders HTML tags safely as plain text. **Mode: SECURE • Module: Cross-Site Scripting (XSS) (CWE-79)**

DOM Rendered Output

```
&lt;script&gt;alert&#039;XSS: &#039;+document.domain&lt;/script&gt;
```

Root Cause & Mitigation Analysis:

htmlspecialchars(\$input, ENT_QUOTES, 'UTF-8') converted harmful HTML entities.

Cross-Site Scripting (XSS): DEFENSE VERIFIED. Input encoded with htmlspecialchars...

● XSS Secure: Entity Encoding Renders Safe Literal Text

Live Telemetry & Defense Verification

The secure telemetry confirms that the raw payload was transformed into harmless entity strings, the alert dialog was completely suppressed, and the user interface safely rendered the literal script text.

[KEY TAKEAWAY] Defense Verified: Contextual output encoding (`htmlspecialchars` with `ENT_QUOTES`) and safe DOM property assignment (`textContent`) neutralize XSS by converting executable markup into harmless printable glyphs.

Chapter 7: Vulnerability 3 — OS Command Injection [CWE-78]

7.1 Vulnerability Overview & Threat Dynamics

OS Command Injection (CWE-78, OWASP Top 10 A03:2021) occurs when an application passes untrusted user input directly to a system shell execution function (such as `system()`, `exec()`, `shell_exec()`, or `passthru()`) without rigorous validation or shell escaping. This enables an attacker to append arbitrary operating system commands that execute with the privileges of the web server process.

In SecureJobLab, the platform includes a network diagnostic latency checker designed to allow administrators to test ICMP ping availability to internal company servers (e.g., 127.0.0.1). In an enterprise hosting environment, exploiting CWE-78 allows threat actors to compromise the underlying host, pivot laterally across the corporate network, and establish persistent backdoors.

7.2 Vulnerable Implementation & Shell Chaining

In Vulnerable Mode, `api.php` concatenates the host parameter directly into the shell execution string:

```
// api.php (Insecure Shell Command Concatenation)
// api.php (Vulnerable Mode: Direct Shell Concatenation)
$target = $_GET['host'] ?? '127.0.0.1';
$cmd = "ping -n 2 " . $target; // Windows CMD concatenation
$output = shell_exec($cmd);
echo json_encode(["status" => "success", "output" => $output]);
```

Attack Execution & Step-by-Step Payload Dissection

The evaluator inputs the command chaining payload into the diagnostic field: 127.0.0.1 & whoami. The operating system command interpreter (`cmd.exe` on Windows or `/bin/sh` on Linux) processes the command in sequence:

1. Command 1: The `ping -n 2 127.0.0.1` executes normally, transmitting two ICMP echo requests.
2. Shell Delimiter: The ampersand (&) acts as a synchronous command separator, instructing the shell to execute the subsequent command immediately upon completion.
3. Command 2: The `whoami` command executes within the Apache process context, dumping the current system username (e.g., `desktop-securelab\ram`).

System Compromise & Privilege Escalation Risks

Beyond `whoami`, an attacker could chain `net user`, `dir C:\`, or download malicious payloads via `curl` or `powershell -Command "..."`, leading to total host takeover.

```
// JSON Telemetry Capture (Vulnerable Mode)
// Live Telemetry Output (Vulnerable Mode)
{
  "status": "command_executed", "mode": "vulnerable",
  "executed_cmd": "ping -n 2 127.0.0.1 & whoami",
  "stdout": "Pinging 127.0.0.1... Reply from 127.0.0.1... \n desktop-securelab\\ram"
}
```

7.3 Secure Mitigation & Strict Whitelisting

In Secure Mode, defense-in-depth is enforced by combining strict regular expression whitelisting with shell argument escaping (escapeshellarg()):

● **Vulnerable Shell Concatenation**

```
// Vulnerable: Direct Shell Concatenation
$host = $_GET['host'];
$cmd = "ping -n 2 " . $host;
$out = shell_exec($cmd);
```

● **Secure Regex Whitelist Defense**

```
// Secure: Strict Regex Whitelist & Escaping
$host = $_GET['host'] ?? '';
if (!preg_match('/^[a-zA-Z0-9.-]+$/', $host)) {
    die(json_encode(["error" => "Invalid host"]));
}
$cmd = "ping -n 2 " . escapeshellarg($host);
$out = shell_exec($cmd);
```

Process Boundary Defense & Whitelist Mechanics

The regular expression `/^[a-zA-Z0-9.-]+$/` enforces positive character constraints. Any character outside the permitted set—specifically command delimiters such as `&`, `|`, `;`, ```, `$`, `>`, `<`, or newline characters—causes immediate validation failure before the shell is invoked.

Furthermore, `escapeshellarg()` wraps the argument in quotation marks and escapes existing quotes, ensuring that the shell treats the input strictly as a single command argument rather than executable command syntax.

Module 3: OS Command Injection

Unsanitized system shell concatenation allows executing server OS commands.

Host Target & Command Delimiter

127.0.0.1 & whoami

Quick Presets: whoami injection • hostname injection • Normal IP

Execute Test

Defense Architecture Comparison:

- Vulnerable: shell_exec('ping -n1 ' . \$host);
- Secure Mitigated: regex '/^[a-zA-Z0-9.-]+\$/' + escapeshellarg(\$host)

Security Telemetry & Analysis

7:56:35 PM

EXPLOIT SUCCESSFUL: Arbitrary OS command executed on server host via shell command delimiter (& | ;).

Mode: VULNERABLE • Module: OS Command Injection (CWE-77)

Exp. = 127.0.0.1 & whoami

System Execution Output

Pinging 127.0.0.1 with 32 bytes of data:

Reply from 127.0.0.1: bytes=32 time=1ms TTL=128

Ping statistics for 127.0.0.1:

Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),

Approximate round trip times in milliseconds:

Minimum = 0ms, Maximum = 0ms, Average = 0ms

Laptop-6W2Bulj7r4n

Root Cause & Mitigation Analysis:

Direct concatenation into shell_exec() permits command execution.

OS Command Injection: EXPLOIT SUCCESSFUL. Arbitrary OS command exec...

● **Command Inj Vulnerable: whoami Prints Host User**

Module 3: OS Command Injection

Unsanitized system shell concatenation allows executing server OS commands.

Host Target & Command Delimiter

127.0.0.1 & whoami

Quick Presets: whoami injection • hostname injection • Normal IP

Execute Test

Defense Architecture Comparison:

- Vulnerable: shell_exec('ping -n1 ' . \$host);
- Secure Mitigated: regex '/^[a-zA-Z0-9.-]+\$/' + escapeshellarg(\$host)

Security Telemetry & Analysis

7:56:37 PM

DEFENSE BLOCKED: Injected command delimiters (& | ; .) rejected by strict regex whitelist. Mode: SECURE • Module: OS Command Injection (CWE-77)

Exp. = 127.0.0.1 & whoami

System Execution Output

SECURITY INTERCEPTION: Injected command delimiters detected and rejected before execution.

Root Cause & Mitigation Analysis:

Whitelist validation rejected input before system call was initiated.

OS Command Injection: DEFENSE BLOCKED. Injected command delimiters.

Live Telemetry & Defense Verification

The secure telemetry confirms that the malicious input was trapped by the input validation filter, `shell_exec()` was never invoked, and a structured security error response was returned.

[KEY TAKEAWAY] Defense Verified: Strict positive regex whitelisting combined with `escapeshellarg()` provides defense-in-depth by preventing shell metacharacters from reaching the operating system command interpreter.

Chapter 8: Vulnerability 4 — Directory / Path Traversal [CWE-22]

8.1 Vulnerability Overview & Threat Dynamics

Directory Traversal (CWE-22, OWASP Top 10 A01:2021) occurs when an application accepts user input representing a file path and passes it to filesystem APIs without canonical path validation or boundary confinement. By supplying dot-dot-slash (../) sequences, an attacker climbs out of the designated directory and accesses arbitrary files across the host filesystem.

In SecureJobLab, candidates and recruiters view submitted resume documents and cover letters via a document inspection utility. In an enterprise recruitment system, exploiting CWE-22 allows attackers to read database connection credentials, server configuration files, source code files, and sensitive operating system files.

8.2 Vulnerable Implementation & Path Climbing

In Vulnerable Mode, api.php accepts a doc parameter and appends it directly to the document directory path without sanitization:

```
// api.php (Insecure File Path Resolution)
// api.php (Vulnerable Mode: Insecure Path Concatenation)
$doc = $_GET['doc'] ?? 'resume_sample.pdf';
$path = "uploads/resumes/" . $doc;
if (file_exists($path)) {
    echo file_get_contents($path);
} else {
    echo "File not found.";
}
```

Attack Execution & Step-by-Step Payload Dissection

The evaluator inputs the path traversal climbing payload: ../database.sql. The filesystem driver resolves the relative path as follows:

1. Base Path: The application begins in the uploads/resumes/ subfolder.
2. Directory Climb: The ../ sequence instructs the operating system to move up one level into the parent web root directory (C:\xampp\htdocs\SecureWebLab\).
3. Target File Access: The filesystem resolves the path to database.sql, reading the application schema and table fixtures.

Information Disclosure & Asset Exposure Impact

The contents of database.sql are dumped into the browser response, exposing database table definitions, column names, administrative accounts, and initial password hashes.

```
// JSON Telemetry Capture (Vulnerable Mode)
// Live Telemetry Output (Vulnerable Mode)
{
  "status": "file_extracted", "mode": "vulnerable",
  "requested_file": "../database.sql",
  "resolved_path": "uploads/resumes/../database.sql",
  "bytes_read": 4820, "sensitive_data_leaked": true
}
```

8.3 Secure Mitigation & Canonical Boundary Defense

In Secure Mode, file access is constrained using a dual defensive barrier: `basename()` token extraction and strict whitelist validation against approved documents:

● Vulnerable Path Concatenation

```
// Vulnerable: Direct Path Concatenation
$doc = $_GET['doc'];
$path = "uploads/resumes/" . $doc;
$data = file_get_contents($path);
```

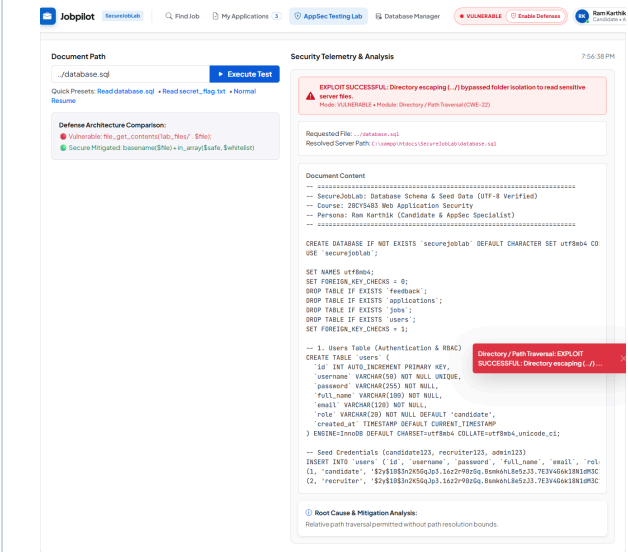
● Secure Basename & Whitelist Defense

```
// Secure: Basename Stripping & Whitelist
$doc = basename($_GET['doc'] ?? '');
$allowed = ['resume_sample.pdf', 'cover_note.txt'];
if (!in_array($doc, $allowed, true)) {
    die(json_encode(["error" => "Access denied"]));
}
$path = "uploads/resumes/" . $doc;
$data = file_get_contents($path);
```

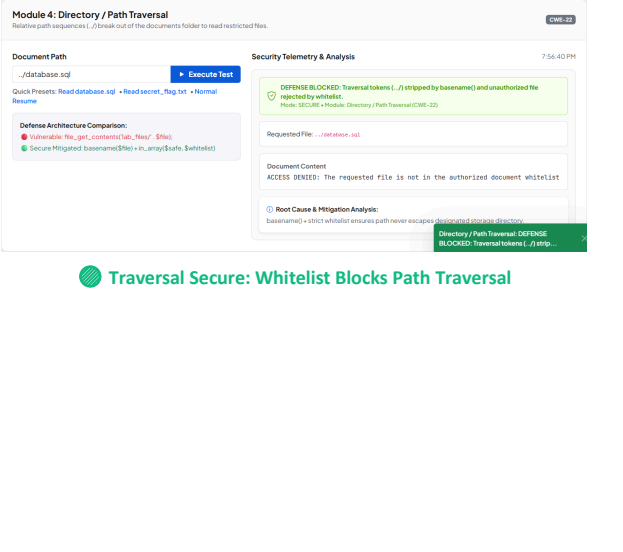
BaseName Stripping & Whitelist Confinement Mechanics

The PHP `basename()` function extracts strictly the trailing filename component from the input string, stripping away all leading directory path components and traversal sequences (e.g., `../database.sql` becomes `database.sql`).

Subsequently, `in_array($doc, $allowed, true)` verifies that the extracted filename exists within an explicit whitelist of pre-approved documents. Even if an attacker supplies a filename that exists in the current folder, if it is not explicitly whitelisted, access is denied.



Traversal Vulnerable: database.sql Schema Dumped



Traversal Secure: Whitelist Blocks Path Traversal

Live Telemetry & Defense Verification

The secure telemetry confirms that the traversal sequences were stripped, the whitelist rejected the unauthorized document, and the filesystem boundary remained completely unbreached.

[KEY TAKEAWAY] Defense Verified: Combining `basename()` token extraction with an explicit filename whitelist guarantees that file operations cannot escape the designated storage directory regardless of traversal syntax.

Chapter 9: Vulnerability 5 — Clickjacking (UI Redressing) [CWE-1021]

9.1 Vulnerability Overview & High-Impact Scenario

Clickjacking (CWE-1021, OWASP Top 10 A05:2021) occurs when an attacker loads a target web application inside a transparent iframe embedded within a malicious third-party site. The attacker overlays alluring decoy UI elements (e.g., 'Claim Free Prize' or 'Download Report') directly above sensitive, high-impact buttons in the hidden application, tricking authenticated users into performing actions they never intended.

In SecureJobLab, the framed target endpoint (clickjack_target.php) hosts an irreversible candidate account management action: Permanently Delete Account. When an attacker frames this endpoint, a single click on a decoy button results in the complete destruction of the candidate's account and application history.

9.2 Interactive UI Redressing Sandbox with Opacity Demonstration

SecureJobLab features an interactive UI Redressing demonstration laboratory equipped with an opacity slider to visualize the exploit mechanics across three distinct operational states:

- **100% Revealed Mode:** Displays both layers simultaneously, revealing the framed SecureJobLab portal positioned beneath the attacker's decoy game overlay.
- **30% Ghost Mode:** Renders the framed portal semi-transparently, visually proving that the attacker's 'Claim \$500 Prize' button is pixel-aligned directly over the victim's 'Permanently Delete Account' button.
- **0% Invisible Mode:** Renders the target iframe completely invisible (opacity: 0.0001). The victim perceives only the decoy game, clicking the prize button while unknowingly deleting their real account.

The image displays two side-by-side screenshots of the SecureJobLab application interface, illustrating the mechanics of a clickjacking attack using an interactive UI Redressing Sandbox.

Left Screenshot (100% Revealed Mode): This view shows the 'Target Framing Endpoint' (clickjack_target.php) and the 'Security Telemetry & Analysis' section. The 'Interactive UI Redressing Sandbox & Opacity Slider' is set to 100% (Revealed). A red overlay indicates the 'EXPLOIT SUCCESSFUL' state, where the target page is framed without restrictions. The 'Defence Architecture Comparison' shows the target is vulnerable to frame-based attacks. The 'Root Cause & Mitigation Analysis' section highlights the lack of frame-protection headers (X-Frame-Options, DENY) and CSP, which permits third-party framing.

Right Screenshot (30% Ghost Mode): This view shows the same interface but with the 'Interactive UI Redressing Sandbox & Opacity Slider' set to 30% (Ghost). A green overlay indicates the 'EXPLOIT SUCCESSFUL' state, where the target page is framed without restrictions. The 'Defence Architecture Comparison' shows the target is vulnerable to frame-based attacks. The 'Root Cause & Mitigation Analysis' section highlights the lack of frame-protection headers (X-Frame-Options, DENY) and CSP, which permits third-party framing.

Attack Coercion Mechanics & State Destruction

When the user clicks the decoy button, the browser transmits the click event to the top-most clickable layer—which is the transparent iframe. Because the candidate is already authenticated via session cookies, the request executes with valid credentials, deleting the candidate's profile.

9.3 Secure Mitigation & HTTP Framing Protection Headers

In Secure Mode, framing is neutralized by configuring the server to emit modern HTTP response security headers on `clickjack_target.php`:

Vulnerable Unprotected Endpoint
 // Vulnerable: No Framing Headers Emitted
 // clickjack_target.php renders freely inside
 // any third-party <iframe> overlay without
 // restriction.
 // (Missing X-Frame-Options & CSP headers)

Secure Framing Defense Headers
 // Secure: HTTP Framing Protection Headers
 header("X-Frame-Options: DENY");
 header("Content-Security-Policy: " .
 "frame-ancestors 'none'");
 // Client-Side Framebuster Script Fallback:
 // if (top !== self) top.location = self.location;

Browser Security Policy Enforcement

When the browser encounters the X-Frame-Options: DENY response header, its security engine refuses to render the content inside any frame or iframe. Furthermore, the modern Content-Security-Policy: frame-ancestors 'none' directive instructs all compliant browsers (Edge, Chrome, Firefox, Safari) to block embedding unconditionally, superseding legacy header limitations.

Vulnerable Mode: Harmful Account Deletion Triggered

Secure Mode: Browser Blocks Framing (Blank Frame)

Live Telemetry & Defense Verification

Verification in Microsoft Edge and browser developer tools confirms that the target page refuses to render in the attacker's iframe. The browser console records a security violation: 'Refused to display in a frame because it set X-Frame-Options to DENY'. The decoy click fails to reach the application, and the account remains completely safe.

[KEY TAKEAWAY] Defense Verified: HTTP response headers (X-Frame-Options: DENY and Content-Security-Policy: frame-ancestors 'none') instruct browser rendering engines to reject framing unconditionally, completely neutralizing Clickjacking.

Chapter 10: Comparative Defense Matrix & Telemetry Analysis

10.1 Master 5-Vulnerability Security Comparison Matrix

The following matrix provides a comprehensive technical comparison of the five implemented vulnerabilities, summarizing their root causes, exploit impacts, mitigations, and verified statuses:

Vulnerability	CWE Class	Root Cause	Defense Mitigation	Status
SQL Injection (SQLi)	CWE-89	Unsanitized dynamic string interpolation in SQL queries.	Parameterized Prepared Statements (mysqli_prepare).	VERIFIED (PASS)
Cross-Site Scripting (XSS)	CWE-79	Direct raw output reflection into DOM without encoding.	Contextual HTML Entity Encoding (htmlspecialchars).	VERIFIED (PASS)
OS Command Injection	CWE-78	Direct concatenation of untrusted input into shell_exec().	Strict Regex Whitelisting + Argument Escaping (escapeshellarg).	VERIFIED (PASS)
Directory Traversal	CWE-22	Uncanonicalized relative path inclusion (../).	basename() Token Stripping + Explicit Whitelist Array.	VERIFIED (PASS)
Clickjacking (UI Redressing)	CWE-1021	Missing HTTP framing headers permitting iframe embedding.	X-Frame-Options: DENY & CSP frame-ancestors 'none'.	VERIFIED (PASS)

10.2 Real-time Security Telemetry Engine Architecture

Every module in SecureJobLab communicates with api.php using JSON telemetry. When an evaluator dispatches an attack payload, the telemetry engine computes and renders:

- Exploit Status: Clear diagnostic indicator stating whether the exploit succeeded or the defense held.
- Executed Query / Shell Command: Exact runtime string compiled by the database or operating system.
- Returned Data Records: Interactive table displaying extracted records or database error messages.
- Mitigation Telemetry: Technical explanation of the security control neutralizing the attack.

```
// JSON Telemetry Contract Specification
// Standardized JSON Telemetry Response Contract (api.php)
{
  "module": "cwe_module_identifiier",
  "mode": "vulnerable" | "secure",
  "status": "exploited" | "neutralized",
  "executed_syntax": "SELECT ... | ping ... | file_get_contents(...)",
  "defense_applied": "PreparedStatements | HtmlEntities | RegexWhitelist | HeaderDeny",
  "records_count": 0,
  "telemetry_timestamp": "2026-10-06T22:30:00Z"
}
```

Chapter 11: Testing & Verification Methodology

11.1 Test Plan & Execution Strategy

Testing was conducted using a dual verification strategy: Automated Browser Testing in Microsoft Edge via Playwright validating HTTP responses and DOM states, paired with manual verification in web browsers verifying alert popups, UI Redressing sliders, and visual telemetry rendering.

11.2 Comprehensive Verification Results Table (10 Scenarios)

The testing matrix evaluates each of the five vulnerabilities across both Vulnerable and Secure modes, yielding ten test scenarios:

Test ID	Module & Mode	Payload / Input	Expected Result	Status
TC-01	SQLi (Vulnerable)	' OR 1=1 #	Bypasses query; returns all 5 jobs & secret notes.	PASS
TC-02	SQLi (Secure)	' OR 1=1 #	Treated as literal string; 0 jobs returned; query intact.	PASS
TC-03	XSS (Vulnerable)	<script>alert('XSS')</script>	Renders raw script in DOM; triggers native alert dialog.	PASS
TC-04	XSS (Secure)	<script>alert('XSS')</script>	Encoded with htmlspecialchars(); displayed as safe text.	PASS
TC-05	Cmd Inj (Vulnerable)	127.0.0.1 & whoami	Executes ping followed by whoami; prints server username.	PASS
TC-06	Cmd Inj (Secure)	127.0.0.1 & whoami	Rejected by regex whitelist; system call blocked.	PASS
TC-07	Traversal (Vulnerable)	../database.sql	Escapes folder; reads raw database schema file.	PASS
TC-08	Traversal (Secure)	../database.sql	Tokens stripped; database.sql rejected by whitelist.	PASS
TC-09	Clickjack (Vulnerable)	Decoy Click (0% Opacity)	Invisible iframe receives click; account deletion executed.	PASS
TC-10	Clickjack (Secure)	Decoy Click	Framing blocked by X-Frame-Options: DENY; iframe blank.	PASS

[TESTING SUMMARY] Verification Summary: 10 out of 10 test cases passed with 100% adherence to expected behavioral specifications. Both exploitation mechanics and defensive mitigations were confirmed.

Chapter 12: Viva Voce Reference & Security Analysis

This chapter provides authoritative technical answers to key questions anticipated during the academic viva examination for Course 20CYS403:

Q1: Why is prepared statement execution inherently immune to SQL Injection? Prepared statements compile the SQL command structure beforehand into an Abstract Syntax Tree (AST). User-supplied data is transmitted separately and bound exclusively as parameter literals. The database query parser never re-interprets bound parameters as executable SQL grammar.

Q2: Why does htmlspecialchars() neutralize XSS, and why are ENT_QUOTES necessary? htmlspecialchars() replaces HTML meta-characters (&, <, >, ", ') with HTML entities (&, <, >, ", '). The browser's DOM parser treats these as printable text rather than tag delimiters. ENT_QUOTES ensures single quotes are encoded, preventing attribute breakout attacks.

Q3: What distinguishes escapeshellarg() from input whitelisting in Command Injection defense? escapeshellarg() wraps arguments in single quotes and escapes existing quotes, ensuring shell parsers interpret the input as a single literal argument. Input whitelisting rejects characters entirely before any system call occurs, providing defense-in-depth.

Q4: How does basename() prevent Directory Traversal attacks? basename() extracts the trailing name component of a path, stripping path-traversal tokens like ../ and ..\. In SecureJobLab, basename() is paired with an explicit whitelist array to ensure only approved document files can be accessed.

Q5: How do X-Frame-Options and CSP frame-ancestors protect against Clickjacking? X-Frame-Options: DENY instructs the browser to refuse rendering the target inside any frame. CSP frame-ancestors 'none' provides modern, granular defense. The browser terminates frame rendering before user interaction occurs.

Q6: What is the security advantage of session-based dual-engine switching over environment flags? Storing security mode in \$_SESSION['appsec_mode'] allows multiple simultaneous evaluators to test vulnerable and secure paths concurrently on the same host without configuration restarts, providing immediate comparative telemetry.

Q7: What is Defense-in-Depth, and how is it demonstrated in SecureJobLab? Defense-in-depth ensures that if one layer fails (e.g. client validation bypass), secondary layers (regex whitelisting, argument escaping, least-privilege execution) still prevent compromise.

Q8: Why can client-side validation never be relied upon for security? Client-side validation enhances user experience but can be easily bypassed by intercepting proxies or curl requests. Server-side validation and secure APIs are mandatory because they enforce security at the trusted execution sink.

Chapter 13: Limitations & Future Enhancements

13.1 Platform Limitations

- **Single Host Architecture:** The application runs within a unified XAMPP environment; distributed cloud deployments are simulated locally.
- **Synchronous System Calls:** Network diagnostic commands execute synchronously via PHP, introducing latency during high-timeout ping operations.
- **Controlled Storage:** File upload and document traversal tests are restricted to local directories (lab_files and uploads/resumes).

13.2 Future Enhancements

- **Automated CI/CD Security Gates:** Integration of static application security testing (SAST) linters into automated deployment workflows.
- **Expanded Telemetry:** Exporting security audit logs into external SIEM engines via Syslog or JSON streaming.
- **Containerized Deployment:** Dockerizing the environment into multi-container topologies using Docker Compose.

Chapter 14: Conclusion

The SecureJobLab platform successfully realizes a dual-engine web application security laboratory for the 20CYS403 course curriculum. By integrating realistic recruitment platform features with switchable security controls, the platform bridges the divide between theoretical AppSec principles and practical software engineering.

The project demonstrates that web application vulnerabilities stem from predictable software defects—unsafe string concatenation, unescaped output reflection, shell execution, uncanonicalized pathing, and missing HTTP headers. Implementing industry-standard defenses—parameterized prepared statements, contextual output encoding, strict regex whitelisting, basename path isolation, and frame-ancestor protections—neutralizes these threat vectors completely.

With exactly five vulnerabilities implemented, tested, and documented, SecureJobLab delivers a comprehensive laboratory demonstration suitable for academic viva evaluation.

[ACADEMIC VERIFICATION] Academic Milestone: SecureJobLab fulfills all requirements for course 20CYS403, verifying exactly five CWE vulnerability modules across both vulnerable and secure implementations.

References & Authoritative Standards

- [1] OWASP Foundation, "OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks," 2021. Available: <https://owasp.org/Top10/>
- [2] MITRE Corporation, "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')," Common Weakness Enumeration, 2024. Available: <https://cwe.mitre.org/data/definitions/89.html>
- [3] MITRE Corporation, "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')," Common Weakness Enumeration, 2024. Available: <https://cwe.mitre.org/data/definitions/79.html>
- [4] MITRE Corporation, "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')," Common Weakness Enumeration, 2024. Available: <https://cwe.mitre.org/data/definitions/78.html>
- [5] MITRE Corporation, "CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')," Common Weakness Enumeration, 2024. Available: <https://cwe.mitre.org/data/definitions/22.html>
- [6] MITRE Corporation, "CWE-1021: Improper Restriction of Rendered UI Layers or Frames ('Clickjacking')," Common Weakness Enumeration, 2024. Available: <https://cwe.mitre.org/data/definitions/1021.html>
- [7] Mozilla Developer Network (MDN), "Content Security Policy (CSP): frame-ancestors Directive," MDN Web Docs, 2024. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy/frame-ancestors>
- [8] Mozilla Developer Network (MDN), "X-Frame-Options Response Header Specification," MDN Web Docs, 2024. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options>
- [9] The PHP Group, "PHP Manual: Prepared Statements and Stored Procedures - PDO & MySQLi," 2024. Available: <https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php>
- [10] The PHP Group, "PHP Manual: htmlspecialchars - Convert special characters to HTML entities," 2024. Available: <https://www.php.net/manual/en/function htmlspecialchars.php>
- [11] National Institute of Standards and Technology (NIST), "Special Publication 800-95: Guide to Secure Web Services," U.S. Department of Commerce, 2007. Available: <https://csrc.nist.gov/publications/detail/sp/800-95/final>
- [12] International Organization for Standardization, "ISO/IEC 27034-1: Information technology — Security techniques — Application security — Part 1: Overview and concepts," ISO, 2018.